Assignment 4
Cmpt 276 – group 1
Dev Tomar : 301548576

Work Distribution

For this assignment, I worked on both identifying and refactoring code smells across different parts of the project.

I identified and or assigned **four code smells** in Mohammed's code and refactored them. These mainly involved improving type safety, reducing parameter complexity, and removing redundant control logic.

Additionally, Daniil suggested **five code smells** in my code, which I refactored across classes such as *Guard*, *MovingEntity*, *Position*, and *Board*. These changes focused on improving encapsulation, reducing duplication, and breaking down complex methods.

Overall, I worked on a total of **nine code smells**, improving the overall readability and maintainability of the codebase.

Code I Refactored (Suggested by Daniil)

I addressed several code smells across *Guard*, *MovingEntity*, *Position*, and parts of *Board* based on the recommendations in *Refactoring.pdf*.

In *MovingEntity*, I simplified the canMoveTo(...) method by removing an unnecessary if/else and returning the boolean expression directly. I also improved encapsulation by making id and position private with appropriate accessors.

In *Board*, I reduced duplication by introducing helper methods such as a coordinate conversion method (toPixel(...)) and consolidating repeated cell-scanning logic used in isWall( ), isOccupied( ), and isSpawnable( ).

In *Guard*, I refactored a long method into smaller methods such as moveTowards( ), improving readability and reducing complexity. I also removed primitive obsession by using the *Position* abstraction instead of raw values. Additionally, I centralized movement behavior in *MovingEntity* through move(...), eliminating duplicated logic.

Code Smells I Suggested and Refactored for Mohammed

The following code smells were identified during the code review and refactored:

- **Primitive Obsession** – Replaced raw character values ('M', 'D', 'S') with an ItemType enum to improve readability and type safety.
- **Switch Statement** – Removed switch-case logic and replaced it with enum-based conditional handling, improving maintainability.
- **Long Parameter List** – Reduced the number of parameters in createItem( ) by grouping related images into a single Image[].
- **Data Clumps** – Combined related image parameters into an array, since they are always used together.

Conclusion

The assignment helped identify and refactor multiple code smells across the project, improving structure, readability, and maintainability. Changes such as better encapsulation, reduced duplication, and simplified logic made the code more efficient and easier to manage. All refactorings were tested to ensure the game's functionality remained intact.